

Lecture 2

lecture-02\ex-1.js

```
1  const y_name = "Pasinee";
2  const score = 80;
3  console.log(`Welcome, ${y_name}! Your score is ${score + 5}.`);
4
5  // OUTPUT : Welcome, Pasinee! Your score is 85.
6
```

lecture-02\ex-2.js

```
1  const y_status = 200;
2
3  console.log("if else:");
4  if (y_status === 200){
5      console.log("OK");
6  }else if (y_status === 400){
7      console.log("ERROR!");
8  }else{
9      console.log("Unknow");
10 }
11 console.log("");
12
13 console.log("switch case :");
14 switch (y_status){
15     case 200:
16         console.log("OK");
17         break;
18     case 400:
19         console.log("ERROR!");
20         break;
21     default:
22         console.log("Unknow");
23 }
24
25 console.log("");
26
27 console.log("Ternary operator :");
28 const message = (y_status === 200) ? "OK!" : "Error!";
29 console.log(message);
30 /** OUTPUT:
31 if else:
32 OK
33
34 switch case :
35 OK
36
37 Ternary operator :
38 OK!
39 */
```

lecture-02\ex-arrow.js

```
1  const a = 5;
2  const b = 10;
3
4  // regular function
5  const add = function (a, b) {
6    return (a + b);
7  };
8  console.log(`The sum of ${a} and ${b} is ${add(a, b)}`);
9
10 // arrow functions
11 const add_1 = (a, b) => (a + b);
12 console.log(`The sum of ${a} and ${b} is ${add_1(a, b)}`);
13
14 // multi arrow line
15 const subtract = (a, b) => {
16   const result = a - b;
17   return result;
18 };
19 console.log(`The difference of ${a} and ${b} is ${subtract(a, b)}`);
20 /*OUTPUT:
21 The sum of 5 and 10 is 15
22 The sum of 5 and 10 is 15
23 The difference of 5 and 10 is -5
24 */
```

lecture-02\ex-censorWord.js

```
1  function censorWord(sentence, word){
2    const regex = new RegExp(word, 'gi');
3    const censorText = sentence.replace(regex, '****');
4    return censorText;
5  }
6
7  const original = "JavaScript is fun, but sometimes javascript can be tricky."
8
9  const cleanPost = censorWord(original, "javascript");
10
11 console.log(cleanPost)
12 /* OUTPUT :
13 **** is fun, but sometimes **** can be tricky.
14 */
```

lecture-02\ex-convertTime.js

```
1  const registrationDate = new Date('2025-12-01T10:00:00');
2  const options = {
3    year: 'numeric',
4    month: 'long',
5    day: 'numeric',
6    timeZone: 'Asia/Bangkok'
7  };
8  const formattedDate = registrationDate.toLocaleDateString('th-TH', options);
9  console.log(`Member Register Date: ${formattedDate}`)
10 /*OUTPUT :
11 Member Register Date: 1 ธันวาคม 2568
12 */
```

lecture-02\ex-function.js

```
1 function findPerimeter(length, width){
2     const perimeter = (length + width) * 2;
3     return perimeter;
4 }
5
6 let result = findPerimeter(5, 10);
7 console.log(`The perimeter is : ${result}`)
8 // OUTPUT : The perimeter is : 30
```

lecture-02\ex-log.js

```
1 function createLogEntry(message){
2     const now = new Date();
3     const timestamp = now.toLocaleTimeString('th-TH');
4     const evenID = Math.random().toString(16).substring(2, 10);
5     const logMessage = message.toUpperCase();
6     return `[${timestamp}] [${evenID}] - ${logMessage}`;
7 }
8
9 const log = createLogEntry("User login successful");
10 console.log(log);
11 // OUTPUT : [23:33:26] [02c9f014] - USER LOGIN SUCCESSFUL
```

lecture-02\ex-math-1.js

```
1 /*
2 Math.random()
3 .toString(36) แปลงสตริงฐาน 36 0-9, a-z
4 .substring(2, 8) ตัด 0. ข้างหน้าออก และเอามา 6 ตัวอักษร
5 */
6
7 function generateTempID() {
8     const randomPart = Math.random().toString(36).substring(2, 8);
9     return randomPart.toUpperCase();
10 }
11
12 const orderID = generateTempID();
13 console.log(`ID : ${orderID}`);
14 // OUTPUT : ID : XWBH1V
```

lecture-02\ex-split.js

```
1 const fullname = " Phasinee Kengklang "
2
3 const cleaneName = fullname.trim();
4 console.log(`Cleane Name: '${cleaneName}'`);
5
6 const nameParts = cleaneName.split(" ");
7 console.log("Name Parts: ",nameParts);
8
9 const finalParts = nameParts.filter(Boolean);
10 console.log("Final Parts:",finalParts);
11
12 const firstname = finalParts[0];
13
14 console.log("Fisrt Name", firstname);
15 /* OUTPUT
16 Cleane Name: 'Phasinee Kengklang'
17 Name Parts: [ 'Phasinee', '', 'Kengklang' ]
18 Final Parts: [ 'Phasinee', 'Kengklang' ]
19 Fisrt Name Phasinee
20 */
```

lecture-02\ex-timing.js

```
1  const startTime = Date.now();
2
3  function someHeavytask(){
4      for (i = 0; i < 100; i++){
5          console.log(i);
6      }
7  }
8
9  const endTime = Date.now();
10 const duration = endTime - startTime;
11 console.log(`function timer: ${duration} milisecon.`)
12 // OUTPUT : function timer: 0 milisecon.
```

Lecture 3

lecture-03\arr_for.js

```
1  const names = ['Justin', 'Sarah', 'Christopher'];
2
3  for (let i = 0; i < names.length; i++){
4      console.log(names[i]);
5  }
6  names.forEach(name => {
7      console.log(name);
8  })
9  names.forEach(myfuction);
10
11 function myfuction(val){
12     console.log(val);
13 }
14 for(let name of names){
15     console.log(name);
16 }
17 /*OUTPUT :
18 Justin
19 Sarah
20 Christopher
21 */
```

lecture-03\arr_map1.js

```
1  const names = ['Tayler', 'Donald', 'Natasha'];
2
3  const announcement = names.map(member => {
4      return member + ' joind the contest.';
5  });
6  console.log(announcement);
7  /* OUTPUT :
8  [
9      'Tayler joind the contest.',
10     'Donald joind the contest.',
11     'Natasha joind the contest.'
12  ]
13  */
```

lecture-03\arr_map2.js

```
1  const kvArr = [
2    {key: 1, value:10},
3    {key: 2, value:20},
4  ];
5
6  const reformatted = kvArr.map(
7    ({key, value}) => ({[key]: value})
8  );
9  console.log(reformatted);
10 // OUTPUT : [ { '1': 10 }, { '2': 20 } ]
```

lecture-03\arr_reduce.js

```
1  const numbers = [15, 16, 17, 18, 19];
2
3  const sum = numbers.reduce((accumulator, currentValue) =>{
4    return accumulator + currentValue;
5  });
6  console.log(sum);
7  // OUTPUT : 85
```

lecture-03\obj_JSON.js

```
1  const bookObj = {
2    title: "Becoming",
3    author: "Michelle Obama",
4    isAvailable: false,
5  }
6
7  const bookJSON = JSON.stringify(bookObj);
8  console.log(bookJSON);
9  console.log(typeof bookJSON);
10
11 const receiveBookObj = JSON.parse(bookJSON);
12 console.log(receiveBookObj.author);
13 console.log(typeof receiveBookObj);
14 /* OUTPUT :
15 {"title":"Becoming","author":"Michelle Obama","isAvailable":false}
16 string
17 Michelle Obama
18 object
19 */
```

lecture-03\obj.js

```
1  const book = {
2    title: "1984",
3    author: "Geoege Orwell",
4    isAvailable: true,
5    checkOut: function(){
6      this.isAvailable = false;
7    },
8    checkIn: function(){
9      this.isAvailable = true;
10   }
11 };
12 console.log(book.isAvailable); // true
13 book.checkOut();
14 console.log(book.isAvailable); // false
15 book.checkIn();
16 console.log(book.isAvailable); // true
```

lecture-03\wk1-1.js

```
1 const furniture = ['Table', 'Chairs', 'Couch'];
2 for(let txt of furniture){
3     for(let t of txt){
4         console.log(t);
5     }
6 }
```



T
a
b
l
e

lecture-03\wk1-2.js

```
1 let values1 = ['Apple', 1, false];
2 let values2 = ['Fries', 2, true, 'Apple'];
3 let values3 = ['Mars', 9, 'Apple'];
4
5 for (item of values1){
6     for(item2 of values2){
7         if(item === item2){
8             for(item3 of values3){
9                 if(item === item3){
10                     console.log(item)
11                 }
12             }
13         }
14     }
15 }
16 console.log();
17 items = values1.filter(item => values2.includes(item) && values3.includes(item));
18 console.log(items);
19 // OUTPUT : Apple
```

lecture-03\wk2-1.js

```
1 const guests = ['Alice', 'Bob', 'Charlie', 'David'];
2 let index = 0;
3 while(index < guests.length){
4     if (guests[index] === 'Charlie'){
5         console.log('Found Charlie!');
6         break;
7     }
8     index++;
9 }
10 // OUTPUT : Found Charlie!
```

lecture-03\wk2-2.js

```
1 const message = "Hello World";
2 const txt = ['a', 'e', 'i', 'o', 'u'];
3
4 for(let name of message.toLowerCase()){
5     if(name === 'a' || name === 'e' || name === 'i' || name === 'o' || name === 'u'){
6         //console.log(name);
7     }
8     if(txt.includes(name) === true){
9         console.log(name);
10     }
11 }
12 // OUTPUT : e o u
```

Lecture 4

lecture-04\class.js

```
1  class Song{
2      constructor(){
3          this.title;
4          this.author;
5      }
6      play(){
7          console.log("Song playing!");
8      }
9  }
10 const mySong = new Song();
11 mySong.play();
12 // OUTPUT : Song playing!
```

lecture-04\constructorclass.js

```
1  class Song {
2      constructor (title, artist){
3          this.title = title;
4          this.artist = artist;
5      }
6  }
7
8  const mySong = new Song("Bohemian Rhapsody", "Queen");
9  console.log(mySong);
10 console.log(typeof mySong);
11 /* OUTPUT : Song { title: 'Bohemian Rhapsody', artist: 'Queen' }
12 object
13 */
```

lecture-04\extendclass.js

```
1  class Media {
2      constructor(info){
3          this.publishDate = info.publishDate;
4          this.name = info.name;
5      }
6  }
7  class Song extends Media{
8      constructor(songData){
9          super(songData);
10         this.artist = songData.artist;
11     }
12 }
13 const mySong = new Song({
14     artist: "Queen",
15     name: "Bohemian Phapsody",
16     publishDate: 1975,
17 });
18
19 console.log(mySong);
20 console.log(mySong.name);
21 /*OUTPUT : Song { publishDate: 1975, name: 'Bohemian Phapsody', artist: 'Queen' }
22 Bohemian Phapsody */
```

lecture-04\errorhandling.js

```
1  function criticalCode (){
2      throw "throwing an exception";
3  }
4
5  function logError(theException){
6      console.log(theException);
7  }
8
9  //Try...Catch
10 console.log("\n*****Try...Catch*****\n");
11
12 try {
13     criticalCode();
14 } catch (ex){
15     console.log("Got an error");
16     logError(ex);
17 }
18
19 //Throwing in Try...Catch
20 console.log("\n*****Throwing in Try...Catch*****\n");
21
22 try{
23     throw "An exception that is thrown every time";
24 }catch (ex){
25     console.log("Got an error");
26     logError(ex);
27 }
28
29 //Try...Catch..Finally
30 console.log("\n*****Try...Catch..Finally*****\n");
31
32 try {
33     criticalCode();
34 } catch(ex){
35     console.log("Got an error");
36     logError(ex);
37 } finally{
38     console.log("Code that always will run");
39 }
40
41 function hello(){
42     console.log("\n*****Throwing Exception*****\n");
43 }
```


lecture-04\exinput.js

```
1  const rl = require('readline').createInterface({
2    input: process.stdin,
3    output: process.stdout
4  });
5  rl.question('Enter your name: ', (name) => {
6    rl.question('Enter your weight (in kg): ', (weight) => {
7      rl.question('Enter your height (in m): ', (height) => {
8
9
10     let bmi = weight / (height * height);
11
12     console.log(`${name}, your BMI is ${bmi.toFixed(2)}`);
13
14     rl.close();
15
16   });
17 });
18 });
19 /*
20 Enter your name: pasinee
21 Enter your weight (in kg): 50
22 Enter your height (in m): 1.6
23 pasinee, your BMI is 19.53
24 */
```

lecture-04\libArea.mjs

```
1  function areaTriangle(b,n){
2    return (b * n) /2;
3  }
4  function areaCircle(r){
5    return Math.PI * r *r;
6  }
7
8  function areaSqr(w, l){
9    return w * l;
10 }
11 export {
12   areaCircle, areaSqr, areaTriangle
13 };
```

lecture-04\mainArea.mjs

```
1  import { areaCircle, areaSqr } from "../libArea.mjs";
2
3  console.log(areaCircle(6));
4  console.log(areaSqr(3, 4));
5  // OUTPUT : 113.09733552923255
6  // OUTPUT : 12
7
```

lecture-04\libHello.js

```
1  const person = {
2      name: "Anirach",
3      weight: 85
4  }
5  function sayHello(){
6      console.log("Hello World!");
7  }
8  function cube(x){
9      return x * x * x;
10 }
11 const add = (a,b) => a+b;
12 const status = true;
13 module.exports = {person, sayHello, cube, add, status}
```

lecture-04\mainHello.js

```
1  const hello = require('./libHello.js')
2  console.log(hello);
3
4  hello.sayHello();
5  console.log(hello.person.name);
6  console.log(hello.cube(3));
7  console.log(hello.add(5,4));
8  console.log(hello.status);
9  /*OUTPUT : {
10     person: { name: 'Anirach', weight: 85 },
11     sayHello: [Function: sayHello],
12     cube: [Function: cube],
13     add: [Function: add],
14     status: true
15  }
16  Hello World!
17  Anirach
18  27
19  9
20  true
21  */
```

lecture-04\staticmethod.js

```
1  class Dog {
2      constructor(name){
3          this.name = name;
4      }
5
6      introduce(){
7          console.log("This is " + this.name + " !");
8      }
9      // A Static method
10     static bark(){
11         console.log("Woof!");
12     }
13 }
14 const myDog = new Dog("Buster");
15 myDog.introduce();
16 Dog.bark();
17 /* OUTPUT :
18 This is Buster !
19 Woof!
20 */
```

lecture-04\trycatch.html

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7  </head>
8  <body>
9      <h2>My First Web Page.</h2>
10     <p>My First Paragraph.</p>
11
12     <p id="demo"></p>
13
14     <script>
15         try{
16             const lalala=5;
17         }catch (err){
18             alert('Error name ' + err.name);
19             alert('Error message ' +err.message);
20             alert('Error stack ' +err.stack);
21
22             alert(err);
23         }
24         const rl = require('readline').createInterface({
25             input: process.stdin,
26             output: process.stdout
27         });
28     </script>
29 </body>
30 </html>
```

code ตัวอย่าง1

mid_term\ex1.js

```
1 function processAndCombineStrings(strings, lengthThreshold) {
2   // 1. แปลงเป็นตัวพิมพ์ใหญ่ และ กรองความยาว
3   let result = strings
4     .map(item => item.toUpperCase())
5     .filter(item => item.length >= lengthThreshold);
6
7   // 2. จัดเรียงข้อมูล (Sort)
8   result.sort((a, b) => {
9     // ฟังก์ชันช่วยนับสระ
10    const countV = (s) => (s.match(/[AEIOU]/g) || []).length;
11    const vA = countV(a); // จำนวนสระของคำแรก
12    const vB = countV(b); // จำนวนสระของคำที่สอง
13    if (vA !== vB) {
14      return vB - vA; // ถ้าสระไม่เท่ากัน เอาตัวที่สระเยอะกว่าขึ้นก่อน
15    }
16    return a.localeCompare(b); // ถ้าสระเท่ากัน เรียงตามตัวอักษร A-Z
17  });
18  // 3. รวมร่างเป็น String เดียวกัน
19  return result.join(" ");
20 }
21
22 // --- ทดสอบตามตัวอย่างในโจทย์ ---
23 const strings = ["apple", "banana", "cherry", "date", "fig", "grape"];
24
25 console.log(processAndCombineStrings(strings, 5));
26 // ผลลัพธ์: "BANANA APPLE GRAPE CHERRY"
27
28 console.log(processAndCombineStrings(strings, 6));
29 // ผลลัพธ์: "BANANA CHERRY"
30
31 console.log(processAndCombineStrings(["hi", "hello", "world"], 2));
32 // ผลลัพธ์: "HELLO WORLD HI"
```

code ตัวอย่าง1

week5\test_ex1.js

```
1  function processAndCombineStrings(strings, lengthThreshold) {
2      const l_fruit = []
3      let c_v = 0
4      const vowels = ['A', 'E', 'I', 'O', 'U']
5
6      const fruit = strings.filter(name => name.length >= lengthThreshold)
7      const f_map = fruit.map(name => name.toLocaleUpperCase())
8
9      f_map.forEach(txt => {
10         c_v = 0
11         for (let char of txt) {
12             if (vowels.includes(char)) {
13                 c_v++;
14             }
15         }
16         l_fruit.push({name: txt, vowels: c_v})
17     });
18
19     l_fruit.sort((a,b) => {
20         if(b.vowels !== a.vowels){
21             return b.vowels - a.vowels;
22         }
23         return a.name.localeCompare(b.name);
24     })
25
26     const result_str = l_fruit.map(obj => obj.name).join(" ")
27     return result_str
28
29 }
30
31
32 const strings = ["apple", "banana", "cherry", "date", "fig", "grape"];
33
34 console.log(processAndCombineStrings(strings, 5));
35 // -> "BANANA APPLE GRAPE CHERRY"
36
37 console.log(processAndCombineStrings(strings, 6));
38 // -> "BANANA CHERRY"
39
40 console.log(processAndCombineStrings(["hi", "hello", "world"], 2));
41 // -> "HELLO WORLD HI"
```

code ตัวอย่าง2

mid_term\ex2.js

```
1  function updateInventory(currentInventory, newInventory) {
2    // 1. นำข้อมูลเก่ามาใส่ใน Map เพื่อให้ค้นหาและอัปเดตได้ง่าย
3    const inventoryMap = new Map();
4    currentInventory.forEach(item => {
5      inventoryMap.set(item.name, item.quantity); // ใส่ของเก่าลงไปก่อน
6    });
7    // 2. วงเล็บของใหม่เพื่ออัปเดตหรือเพิ่ม
8    newInventory.forEach(newItem => {
9      if (inventoryMap.has(newItem.name)) {
10        // ถ้ามีชื่อนี้อยู่แล้ว ให้บวกจำนวนเพิ่มเข้าไป
11        const currentQty = inventoryMap.get(newItem.name);
12        inventoryMap.set(newItem.name, currentQty + newItem.quantity);
13      } else {
14        // ถ้ายังไม่มี ให้เพิ่มเข้าไปใหม่
15        inventoryMap.set(newItem.name, newItem.quantity);
16      }
17    });
18    // 3. แปลง Map กลับเป็น Array of Objects และเรียงตามชื่อ (A-Z)
19    const result = [];
20    inventoryMap.forEach((quantity, name) => {
21      result.push({ name: name, quantity: quantity });
22    });
23    return result.sort((a, b) => a.name.localeCompare(b.name));
24  }
25
26  const currentInv = [
27    { name: 'item1', quantity: 10 },
28    { name: 'item2', quantity: 5 },
29    { name: 'item3', quantity: 8 }
30  ];
31  const newInv = [
32    { name: 'item1', quantity: 5 },
33    { name: 'item4', quantity: 7 },
34    { name: 'item2', quantity: 10 }
35  ];
36  console.log(updateInventory(currentInv, newInv));
37  /** OUTPUT : [
38    { name: 'item1', quantity: 15 },
39    { name: 'item2', quantity: 15 },
40    { name: 'item3', quantity: 8 },
41    { name: 'item4', quantity: 7 }
42  ]*/
```

code ตัวอย่าง2

week5\test_ex2.js

```
1  /**
2   *
3   * @param {Array} currentInventory
4   * @param {Array} newInventory
5   */
6
7  function updateInventory(currentInventory, newInventory){
8
9      newInventory.forEach(name => {
10         const find_name = currentInventory.find(val => val.name === name.name)
11
12         if(find_name){
13             find_name.quantity += name.quantity
14         }else{
15             currentInventory.push({name: name.name, quantity:name.quantity})
16         }
17     })
18     return currentInventory
19 }
20
21
22 const currentInventory = [
23     { name: 'item1', quantity: 10 },
24     { name: 'item2', quantity: 5 },
25     { name: 'item3', quantity: 8 }
26 ];
27
28 const newInventory = [
29     { name: 'item1', quantity: 5 },
30     { name: 'item4', quantity: 7 },
31     { name: 'item2', quantity: 10 }
32 ];
33
34 console.log(updateInventory(currentInventory, newInventory));
```


code ตัวอย่าง3

mid_term\ex3.js

```
1 function findLongestRepeatedSubstring(inputStr) {
2   let longest = "";
3   const n = inputStr.length;
4
5   // 1. วงลูปหาจุดเริ่มต้น (i) และความยาวของซับสตริง (len)
6   for (let i = 0; i < n; i++) {
7     for (let len = 1; i + len <= n; len++) {
8       // ตัดข้อความย่อยออกมา
9       let sub = inputStr.substring(i, i + len);
10
11       // 2. เช็คว่ซับสตริงนี้ปรากฏที่ "ตำแหน่งอื่น" ในข้อความหรือไม่
12       // โดยเริ่มหาหลังจากตำแหน่งปัจจุบัน (i + 1)
13       if (inputStr.indexOf(sub, i + 1) !== -1) {
14
15         // 3. ถ้าเจอซ้ำ และยาวกว่าตัวที่เคยเก็บไว้ ให้บันทึกเป็นตัวใหม่
16         if (sub.length > longest.length) {
17           longest = sub;
18         }
19       }
20     }
21   }
22   return longest;
23 }
24
25 console.log(findLongestRepeatedSubstring("banana")); // "ana"
26 console.log(findLongestRepeatedSubstring("abcd")); // ""
27 console.log(findLongestRepeatedSubstring("ababa")); // "aba"
```


mid_term\ex4.js

```
1 function wordFrequency(text) {
2     let num_word = {};
3     let new_txt = text.replace(/[!,.]/g, "").toLocaleLowerCase()
4     new_txt = new_txt.split(" ")
5     for (let x of new_txt) {
6         if (num_word[x]) {
7             num_word[x] = num_word[x] + 1;
8         } else {
9             num_word[x] = 1;
10        }
11    }
12    return num_word;
13 }
14
15 console.log(wordFrequency("Hello world! Hello everyone."));
16 console.log(wordFrequency("This is a test. This test is easy."));
17 console.log(wordFrequency("Python is fun. Fun fun fun!"));
18 console.log(wordFrequency("One word, one word."));
19
20 /* OUTPUT
21 { hello: 2, world: 1, everyone: 1 }
22 { this: 2, is: 2, a: 1, test: 2, easy: 1 }
23 { python: 1, is: 1, fun: 4 }
24 { one: 2, word: 2 }
25 */
```

mid_term\ex5.js

```
1 function findSingleOccurrenceNumbers(numbers) {
2     let num = {};
3     let result = []
4     for(let x of numbers){
5         if(num[x]){
6             num[x] += 1
7         }else{
8             num[x] = 1
9         }
10    }
11    Object.keys(num).forEach(n =>{
12        if(num[n] === 1){
13            result.push(n)
14        }
15    })
16    return result;
17 }
18
19 console.log(findSingleOccurrenceNumbers([4, 5, 6, 4, 7, 5, 8])); // [ '6', '7', '8' ]
20 console.log(findSingleOccurrenceNumbers([1, 2, 2, 3, 3, 4, 4])); // [ '1' ]
21 console.log(findSingleOccurrenceNumbers([1, 1, 1, 1, 1])); // []
```

HW - 01

HW\HW-01.js

```
1  function countString(input, type) {
2      if (typeof input !== 'string') {
3          throw new Error("Invalid parameter");
4      }
5      if(input.trim() === ""){
6          return 0;
7      }
8      const word = input.trim().split(" ");
9
10     if(type.toLowerCase() == 'w'){
11         return word.length;
12     }
13     }else if(type.toLowerCase() == 'c'){
14         let sum_txt = 0
15         word.forEach(txt => {
16             sum_txt += txt.length
17         })
18         return sum_txt;
19     }
20     }else if(type.toLowerCase() == 'v'){
21         let vol_sum = 0;
22         const txt_vowel = ['a', 'e', 'i', 'o', 'u'];
23         word.forEach(meassge =>{
24             for(let letter of meassge.toLowerCase()){
25                 if(txt_vowel.includes(letter) === true){
26                     vol_sum += 1;
27                 }
28             }
29         })
30         return vol_sum;
31     }else{
32         throw new Error("Invalid parameter");
33     }
34 }
35
36 // Example usage
37 console.log("Word count:", countString("Hello world, how are you?", "w")); // Output: 5
38 console.log("Character count:", countString("Hello world, how are you?", "c")); // Output: 21
39 console.log("Vowel count:", countString("Hello world, how are you?", "v")); // Output: 8
40
```

HW - 02

HW\HW-02.js

```
1  const library = {
2    books: [],
3
4    addBook: function (book) {
5      // TODO: เขียนโค้ดสำหรับเพิ่มหนังสือ
6      const exists = this.books.some(b => b.title === book.title);
7
8      if (exists) {
9        console.log(`The book "${book.title}" already exists in the library.`);
10     } else {
11       this.books.push(book);
12     }
13   },
14
15   removeBook: function (title) {
16     // TODO: เขียนโค้ดสำหรับลบหนังสือตามชื่อ
17     this.books = this.books.filter(book => book.title !== title);
18   },
19
20   listBooks: function () {
21     // TODO: เขียนโค้ดสำหรับแสดงรายชื่อหนังสือทั้งหมด
22     this.books.forEach((book, index) => {
23       const status = book.isRead ? "อ่านแล้ว" : "ยังไม่ได้อ่าน";
24       console.log(`${index + 1}. ชื่อ: ${book.title}, ผู้แต่ง: ${book.author}, ปีที่พิมพ์: ${book.year}, สถานะ: ${status}`);
25     });
26   },
27
28   getUnreadBooks: function () {
29     // TODO: เขียนโค้ดสำหรับส่งคืนหนังสือที่ยังไม่ได้อ่าน
30     return this.books.filter(book => !book.isRead);
31   }
32 };
33
34
35 library.addBook({ title: "The Great Gatsby", author: "F. Scott Fitzgerald", year: 1925, isRead: true });
36 library.addBook({ title: "To Kill a Mockingbird", author: "Harper Lee", year: 1960, isRead: false });
37 library.addBook({ title: "1984", author: "George Orwell", year: 1949, isRead: true });
38
39 library.listBooks();
40 /* Output:
41 1. ชื่อ: The Great Gatsby, ผู้แต่ง: F. Scott Fitzgerald, ปีที่พิมพ์: 1925, สถานะ: อ่านแล้ว
42 2. ชื่อ: To Kill a Mockingbird, ผู้แต่ง: Harper Lee, ปีที่พิมพ์: 1960, สถานะ: ยังไม่ได้อ่าน
43 3. ชื่อ: 1984, ผู้แต่ง: George Orwell, ปีที่พิมพ์: 1949, สถานะ: อ่านแล้ว
44 */
45
46 console.log(library.getUnreadBooks());
47 // Output: [ { title: "To Kill a Mockingbird", author: "Harper Lee", year: 1960, isRead: false } ]
48
49 library.removeBook("1984");
50 library.listBooks();
51 /* Output:
52 1. ชื่อ: The Great Gatsby, ผู้แต่ง: F. Scott Fitzgerald, ปีที่พิมพ์: 1925, สถานะ: อ่านแล้ว
53 2. ชื่อ: To Kill a Mockingbird, ผู้แต่ง: Harper Lee, ปีที่พิมพ์: 1960, สถานะ: ยังไม่ได้อ่าน
54 */
```

mid2.js

```

1 function findHighestTotal(sales) {
2   let totals = {}; // ตัวสะสมยอดเงินแยกตามหมวดหมู่
3   for (let item of sales) {
4     // 1. คำนวณเรดรวมของรายการนี้ (ราคา x จำนวน)
5     let itemTotal = item.price * item.quantity;
6
7     // 2. จัดกลุ่มและสะสมยอดเงิน (ใช้หมวดหมู่เป็น Key)
8     if (totals[item.category]) {
9       totals[item.category] += itemTotal;
10    } else {
11      totals[item.category] = itemTotal;
12    }
13  }
14  // --- ส่วนแสดงราคาของแต่ละหมวดหมู่ (เพื่อให้เราตรวจสอบได้) ---
15  console.log("สรุปยอดขายแยกตามหมวดหมู่:", totals);
16
17  // 3. หาว่า Key ไหน (หมวดหมู่ไหน) ที่มีมูลค่าเงินสูงที่สุด
18  let maxCategory = "";
19  let maxAmount = 0;
20
21  for (let cat in totals) {
22    if (totals[cat] > maxAmount) {
23      maxAmount = totals[cat];
24      maxCategory = cat;
25    }
26  }
27  return "หมวดที่ขายได้มากที่สุดคือ: " + maxCategory + " (รวม " + maxAmount + " บาท)";
28 }
29 // --- ตัวอย่างข้อมูล ---
30 const salesData = [
31   { category: "Electronics", price: 100, quantity: 2 }, // ยอด: 200
32   { category: "Food", price: 50, quantity: 5 },          // ยอด: 250
33   { category: "Electronics", price: 100, quantity: 1 }  // ยอด: 100
34 ];
35
36 console.log(findHighestTotal(salesData));

```

mid3.js

```

1 function findLongestRepeatedSubstring(words) { // <--- รับค่าเดียวที่เป็น Array
2   let countMap = {}; // ใช้ Object เป็น "สมุดจด" บันทึกจำนวนครั้งที่เจอ
3   let result = [];
4   // 1. วนลูปนับจำนวนคำ
5   // words ในที่นี้คือ Array ที่ส่งเข้ามา เช่น ["banana", "som", "som"]
6   for (let word of words) {
7     let upperWord = word.toUpperCase(); // แปลงเป็นตัวพิมพ์ใหญ่เพื่อให้ "som" กับ "Som" คือคำเดียวกัน
8     if (countMap[upperWord]) {
9       // ถ้าเคยจดชื่อนี้ไว้แล้ว ให้บวกเพิ่มอีก 1
10      countMap[upperWord] = countMap[upperWord] + 1;
11    } else {
12      // ถ้ายังไม่เคยเห็นชื่อนี้ ให้เริ่มนับที่ 1
13      countMap[upperWord] = 1;
14    }
15  }
16  // 2. ตรวจสอบสมุดจด (Object) เพื่อหาตัวที่ซ้ำ
17  for (let key in countMap) {
18    if (countMap[key] > 1) {
19      // ถ้าค่านั้นมีจำนวน (Value) มากกว่า 1 ให้เก็บลงตะกร้าผลลัพธ์
20      result.push([" " + key + ", " + countMap[key] + ""]);
21    }
22  }
23  // 3. ส่งผลลัพธ์กลับ
24  if (result.length === 0) {
25    return "[]";
26  } else {
27    // ใช้ .sort() เพื่อเรียงลำดับ A-Z และ .join(",") เพื่อเอามาต่อกันเป็นสายยาว
28    return result.sort().join(",");
29  }
30 }
31 // --- วิธีการเรียกใช้งาน (สำคัญ: ต้องใส่ [ ] ครอบข้อมูล) ---
32 const myData = ["banana", "Som", "som", "banana", "app"];
33 console.log(findLongestRepeatedSubstring(myData));
34 // Output: "[BANANA,2],[SOM,2]"
35

```

[PROBLEMS](#)
[OUTPUT](#)
[DEBUG CONSOLE](#)
[TERMINAL](#)
[PORTS](#)
[Filter](#)

[Running] node "c:\Users\LEGION\Desktop\Fullstack\mid\mid2.js"
 สรุปยอดขายแยกตามหมวดหมู่: { Electronics: 300, Food: 250 }
 หมวดที่ขายได้มากที่สุดคือ: Electronics (รวม 300 บาท)

[Done] exited with code=0 in 0.088 seconds

mid4.js

```
1 function findLongestRepeatedSubstring(words) {
2   let countMap = {};
3   let result = [];
4   // 1. การนับจำนวนคำ (Counting)
5   for (let word of words) {
6     // ลบสัญลักษณ์พิเศษและแปลงเป็นตัวพิมพ์ใหญ่
7     let cleanWord = word.replace(/[^a-zA-Z0-9]/g, "").toUpperCase();
8
9     if (cleanWord !== "") {
10      if (countMap[cleanWord]) {
11        countMap[cleanWord] = countMap[cleanWord] + 1;
12      } else {
13        countMap[cleanWord] = 1;
14      }
15    }
16  }
17  // 2. การสร้างผลลัพธ์คำที่ซ้ำ (Filtering & Expanding)
18  for (let key in countMap) {
19    if (countMap[key] > 1) {
20      // ถ้าคำนั้นซ้ำ ให้วนลูปใส่ [คำ] ลงในตะกร้าตามจำนวนครั้งที่เจอจริง
21      for (let i = 0; i < countMap[key]; i++) {
22        result.push "[" + key + "]";
23      }
24    }
25  }
26  // 3. การจัดเรียงและส่งค่ากลับ (Sorting & Joining)
27  return result.length === 0 ? "[]" : result.sort().join(",");
28 }
29 // --- การเรียกใช้งานที่ถูกต้อง (ต้องใส่ [ ] ครอบ) ---
30 const inputData = ["banana!", "Som", "som?", "banana"];
31 console.log(findLongestRepeatedSubstring(inputData));
32 // Output: "[BANANA],[BANANA],[SOM],[SOM]"
```

mid5.js

```
1 function analyzeEmployeeID(ids) { // <--- รับค่าเดียวที่เป็น Array ของรหัสพนักงาน
2   let result = [];
3
4   for (let rawId of ids) {
5     // 1. ล้างอักขระพิเศษ (ยกเว้น _) และแปลงเป็นพิมพ์ใหญ่
6     // เช่น "IT_2020_099!" จะกลายเป็น "IT_2020_099"
7     let cleanId = rawId.replace(/[^a-zA-Z0-9_]/g, "").toUpperCase();
8
9     // 2. แยกส่วนประกอบด้วย "_" (ใช้ .split())
10    // ผลลัพธ์ที่ได้จะเป็น Array ย่อย เช่น ["IT", "2020", "099"]
11    let parts = cleanId.split("_");
12    // ตรวจสอบว่ารหัสมี 3 ส่วนประกอบครบถ้วน (แผนก_ปี_ลำดับ)
13    if (parts.length === 3) {
14      let dept = parts[0];
15      let year = parseInt(parts[1]); // แปลงจากสตริง "2020" เป็นตัวเลข 2020
16      let empNum = parts[2];
17
18      // 3. ตรวจสอบเงื่อนไขตามโจทย์: แผนก IT หรือ SALES และต้องเป็นปี 2020 ขึ้นไป
19      if ((dept === "IT" || dept === "SALES") && year >= 2020) {
20        result.push "[" + dept + ": " + empNum + "]";
21      }
22    }
23  }
24  // 4. ส่งคืนผลลัพธ์
25  if (result.length === 0) {
26    return "No matching records";
27  } else {
28    return result.join(", ");
29  }
30 }
31 // --- การเรียกใช้งานที่ถูกต้อง (ต้องมี [ ] ครอบ) ---
32 const data = ["it_2023_001", "hr_2021_005", "Sales_2019_010", "IT_2020_099!"];
33 console.log(analyzeEmployeeID(data));
34 // Output: "[IT: 001], [IT: 099]"
```


mid6.js

```
1 function calculateGrades(students) {
2   let summary = [];
3   let passCount = 0;
4
5   for (let student of students) {
6     let name = student.name;
7     let score = student.score;
8     // 1. ตรวจสอบเงื่อนไขคะแนน (0-100)
9     if (score >= 0 && score <= 100) {
10      let grade = "";
11      // 2. ตัดเกรดตามช่วงคะแนน
12      if (score >= 80) {
13        grade = "A";
14      } else if (score >= 50) {
15        grade = "PASS";
16      } else {
17        grade = "FAIL";
18      }
19      // 3. จัดรูปแบบข้อความและนับจำนวนคนสอบผ่าน
20      summary.push(name + " (" + grade + ")");
21
22      if (grade !== "FAIL") {
23        passCount++;
24      }
25    }
26  }
27  // 4. คืนค่าเป็น Object ตามโจทย์สั่ง
28  return {
29    summary: summary,
30    passCount: passCount
31  };
32 }
33 // --- ตัวอย่างการใช้งาน ---
34 const studentsData = [
35   { name: "Somsak", score: 85, group: "A" },
36   { name: "Wichai", score: 45, group: "B" },
37   { name: "Anna", score: 120, group: "A" }, // ข้อมูลผิดพลาด (ข้าม)
38   { name: "Joy", score: 55, group: "B" }
39 ];
40
41 const result = calculateGrades(studentsData);
42 console.log(result);
43
44 /* Expected Output:
45 {
46   summary: ["Somsak (A)", "Wichai (FAIL)", "Joy (PASS)"],
47   passCount: 2
48 }
49 */
```

mid7.js

```
1 function findLongestRepeatedSubstring(inputStr) {
2   let countMap = {};
3   let result = [];
4
5   // 1. แปลงประโยคเป็นตัวพิมพ์เล็ก และแยกคำด้วยช่องว่าง (Split)
6   // ใช้ .split(" ") เพื่อแยก "Hello world" ให้กลายเป็น ["hello", "world"]
7   let words = inputStr.toLowerCase().split(" ");
8
9   for (let word of words) {
10    // ล้างอักขระพิเศษเพื่อไว้ (เช่นเครื่องหมายคำถามหรือจุด)
11    let cleanWord = word.replace(/^[a-z0-9]/g, "");
12
13    if (cleanWord !== "") {
14      if (countMap[cleanWord]) {
15        countMap[cleanWord] += 1;
16      } else {
17        countMap[cleanWord] = 1;
18      }
19    }
20  }
21  // 2. จัดรูปแบบการแสดงผลให้เป็น 'word': count
22  for (let key in countMap) {
23    result.push("'" + key + "': " + countMap[key]);
24  }
25  // 3. ส่งคืนในรูปแบบสตริงที่มี [ ] ครอบตามตัวอย่างของคุณ
26  return "[" + result.join(", ") + "];";
27 }
28 // --- ทดสอบการใช้งาน ---
29 console.log(findLongestRepeatedSubstring("Hello world Hello everyoun"));
30 // Output: "[ 'hello': 2, 'world': 1, 'everyoun': 1 ]"
```

mid8.js

```
1 function findUniqueWords(inputStr) {
2   let result = [];
3
4   // 1. เตรียมข้อมูล: แปลงเป็นพิมพ์เล็กและแยกเป็น Array
5   let words = inputStr.toLowerCase().split(" ");
6
7   // 2. วนลูปเพื่อเช็คทีละคำ
8   for (let i = 0; i < words.length; i++) {
9     let currentWord = words[i].replace(/^[a-z0-9]/g, ""); // ล้างอักขระพิเศษ
10
11     if (currentWord !== "") {
12       // ตรวจสอบว่าตำแหน่งแรกทีเจอ และตำแหน่งสุดท้ายที่เจอ คือตำแหน่งเดียวกันไหม
13       // ถ้าตำแหน่งเดียวกัน (indexOf === lastIndexOf) แปลว่า "ไม่ซ้ำกับใครเลย"
14       if (words.indexOf(words[i]) === words.lastIndexOf(words[i])) {
15         result.push("'" + currentWord + "'");
16       }
17     }
18   }
19   // 3. แสดงผลลัพธ์
20   return result.length === 0 ? "[]" : "[" + result.join(", ") + "];";
21 }
22 // --- ทดสอบการใช้งาน ---
23 console.log(findUniqueWords("Hello world Hello everyone"));
24 // Output: "[ 'world', 'everyone' ]"
25 // อธิบาย: 'hello' ซ้ำจึงไม่แสดง, ส่วน 'world' และ 'everyone' มีแค่ตัวเดียวจึงแสดงผล
```

mid9.js

```
1 function findUniqueWords(...words) { // ใช้ ... เพื่อรับค่ากี่ค่าก็ได้
2   let result = [];
3
4   // 1. ล้างข้อมูลและทำให้เป็นมาตรฐานเดียวกัน (ตัวพิมพ์เล็ก)
5   // หมายถึง: words ในที่นี้จะเป็น Array อยู่แล้วจาก ...
6   let cleanWords = words.map(w => w.toLowerCase().replace(/^[a-z0-9]/g, ""));
7
8   // 2. วนลูปเช็คทีละคำใน Array cleanWords
9   for (let i = 0; i < cleanWords.length; i++) {
10    let currentWord = cleanWords[i];
11
12    if (currentWord !== "") {
13      // เช็คตำแหน่งแรก (indexOf) และตำแหน่งสุดท้าย (lastIndexOf) ตรงกันไหม
14      // ถ้าตรงกัน แสดงว่ามีคำนั้นโผล่มาแค่ 'ครั้งเดียว' ในชุดข้อมูล
15      if (cleanWords.indexOf(currentWord) === cleanWords.lastIndexOf(currentWord)) {
16        result.push("" + currentWord + "");
17      }
18    }
19  }
20
21  // 3. แสดงผลลัพธ์
22  return result.length === 0 ? "[]" : "[" + result.join(", ") + ""];
23 }
24
25 // --- ทดสอบการใช้งาน ---
26 console.log(findUniqueWords("banana", "Som", "som", "banana", "app"));
27 // Output: ["'app'"]
```

mid9.js

```
1 function findUniqueWords(words) { // <--- รับค่าเดียวที่เป็น Array
2   let result = [];
3
4   // 1. ล้างข้อมูล (Data Cleansing)
5   // ใช้ .map() เพื่อเดินไปหาทุกคำใน Array 'words' แล้วทำ 2 อย่าง:
6   // - .toLowerCase(): ทำให้เป็นพิมพ์เล็ก (เพื่อคนจะได้เห็น "Som" และ "som" เป็นคำเดียวกัน)
7   // - .replace(): ลบสัญลักษณ์พิเศษออก
8   let cleanWords = words.map(w => w.toLowerCase().replace(/^[a-z0-9]/g, ""));
9
10  // 2. ตรวจสอบความซ้ำ (Uniqueness Logic)
11  for (let i = 0; i < cleanWords.length; i++) {
12    let currentWord = cleanWords[i];
13
14    if (currentWord !== "") {
15      // เทคนิคสำคัญ: ถ้าเลขตำแหน่งแรกเจอ (indexOf)
16      // ตรงกับเลขตำแหน่งสุดท้ายเจอ (lastIndexOf)
17      // แสดงว่าในทั้งรายการนั้น... มีคำนี้อยู่แค่ "ตัวเดียว" เท่านั้น
18      if (cleanWords.indexOf(currentWord) === cleanWords.lastIndexOf(currentWord)) {
19        result.push("" + currentWord + "");
20      }
21    }
22  }
23
24  // 3. การแสดงผล (Output)
25  // ใช้ Ternary Operator (เงื่อนไขแบบสั้น) เช็คถ้าค่าจะกรำวางให้คืน []
26  return result.length === 0 ? "[]" : "[" + result.join(", ") + ""];
27 }
28
29 // --- วิธีเรียกใช้งานที่ถูกต้อง (ห้ามลืมใส่ก้ามปู [ ] ) ---
30 console.log(findUniqueWords(["banana", "Som", "som", "banana", "app"]));
31 // Output: ["'app'"]
```

mid10.js

```
1 function findUniqueNumbers(nums) {
2   let result = [];
3   for (let i = 0; i < nums.length; i++) {
4     if (nums.indexOf(nums[i]) === nums.lastIndexOf(nums[i])) {
5       result.push(nums[i]);
6     }
7   }
8   return result;
9 }
10 // การเรียกใช้: ต้องมี [ ]
11 console.log(findUniqueNumbers([123, 123, 456, 456, 789])); // Output: [789]
```


mid11.js

```
1 function findDuplicates(nums) { // <--- รับค่าเดียวที่เป็น Array
2   let duplicates = [];
3
4   for (let i = 0; i < nums.length; i++) {
5     let currentNum = nums[i];
6
7     // 1. ตรวจสอบ indexOf VS lastIndexOf
8     // ถ้า "ตำแหน่งแรกเจอ " ไม่เท่ากับ "ตำแหน่งสุดท้ายเจอ"
9     // แปลว่าเลขนั้นโผล่มามากกว่า 1 ครั้ง (คือมันซ้ำนั่นเอง)
10    if (nums.indexOf(currentNum) !== nums.lastIndexOf(currentNum)) {
11
12      // 2. ป้องกันผลลัพธ์ซ้ำซ้อน
13      // เช็คดูในดู duplicates มีเลขนี้หรือยัง ถ้ายังไม่มี (!) ถึงจะเก็บใส่ดู
14      if (!duplicates.includes(currentNum)) {
15        duplicates.push(currentNum);
16      }
17    }
18  }
19
20  // 3. การส่งผลลัพธ์แบบเงื่อนไข (Ternary Operator)
21  return duplicates.length === 0 ? "No duplicates found" : duplicates;
22 }
23
24 // --- วิธีเรียกใช้งานที่ถูกต้อง (ต้องใส่ [ ] ครบตัวเลข) ---
25 const myNumbers = [123, 123, 456, 456, 789];
26 console.log(findDuplicates(myNumbers));
27 // Output: [123, 456]
```

test3.js

```
1 function findLongestRepeatedSubstring(inputStr) {
2   let longest = "";
3
4   // วนลูปเพื่อตัดแบ่งข้อความเป็นส่วนย่อยๆ ทุกขนาด
5   for (let i = 0; i < inputStr.length; i++) {
6     for (let j = i + 1; j <= inputStr.length; j++) {
7       let sub = inputStr.substring(i, j);
8
9       // เช็คว่ามีซ้ำที่ตำแหน่งอื่นไหม
10      let first = inputStr.indexOf(sub);
11      let second = inputStr.indexOf(sub, first + 1);
12
13      // ถ้าเจอซ้ำ และค่านี้ยาวกว่าที่เคยเจอมา ให้บันทึกไว้
14      if (second !== -1 && sub.length > longest.length) {
15        longest = sub;
16      }
17    }
18  }
19
20  return longest;
21 }
22
23 // --- ตัวอย่างการใ้ใช้งานตามรูปภาพ ---
24 console.log(findLongestRepeatedSubstring("banana")); // Output: "ana"
25 console.log(findLongestRepeatedSubstring("abcd")); // Output: ""
26 console.log(findLongestRepeatedSubstring("ababa")); // Output: "aba"
```

test10.js

```
1 function findLongestRepeatedNumberSequence(numbers) {
2   let groups = {};
3
4   for (let rawStr of numbers) {
5     // 1. ล้างช่องว่าง และแปลงเป็น Array ของตัวเลขเพื่อจัดเรียง
6     // เช่น "321" -> "123"
7     let cleanStr = rawStr.trim();
8     if (cleanStr === "") continue;
9
10    // สร้าง "Key" โดยการเรียงลำดับตัวเลขภายในสตริง
11    let sortedKey = cleanStr.split('').sort().join('');
12
13    // 2. ตรวจสอบใน Object groups
14    if (!groups[sortedKey]) {
15      groups[sortedKey] = []; // ถ้ายังไม่มีกลุ่มนี้ ให้สร้าง Array ว่างรอไว้
16    }
17
18    // 3. แปลงสตริงเป็นตัวเลข (Number) แล้วเก็บเข้ากลุ่ม
19    groups[sortedKey].push(Number(cleanStr));
20  }
21
22  // 4. ดึงเอาเฉพาะค่าในกลุ่ม (Values) ออกมาเป็น Array ช้อน Array
23  return Object.values(groups);
24 }
25
26 // --- ทดสอบการใช้งาน ---
27 const nums1 = ["123", "321", " 456", "654", "546", "788"];
28 console.log("Output 1:", findLongestRepeatedNumberSequence(nums1));
29 // Output: [[123, 321], [456, 654, 546], [788]]
```

test2.js

```
1 function processAndCombineStrings(strings, lengthThreshold) {
2   // 1. เตรียมที่เก็บข้อมูลที่แปลงเป็นคำพิมพ์ใหญ่และกรองความยาวแล้ว
3   let filteredStrings = [];
4
5   for (let s of strings) {
6     let upperCaseWord = s.toUpperCase(); // แปลงเป็นคำพิมพ์ใหญ่
7     if (upperCaseWord.length >= lengthThreshold) {
8       filteredStrings.push(upperCaseWord); // ถ้าความยาวถึงที่กำหนด ให้เก็บใส่ Array
9     }
10  }
11  // 2. ฟังก์ชันช่วยนับสระแบบอ่านง่าย
12  function countVowels(word) {
13    let vowels = word.match(/[AEIOU]/g); // หาตัวที่เป็น A, E, I, O, U
14    if (vowels === null) {
15      return 0; // ถ้าไม่เจอสระเลย ให้เป็น 0
16    }
17    return vowels.length; // ถ้าเจอ ให้ส่งกลับมว่าเจอกี่ตัว
18  }
19  // 3. จัดเรียงข้อมูลใน Array
20  filteredStrings.sort((a, b) => {
21    let countA = countVowels(a);
22    let countB = countVowels(b);
23    if (countA !== countB) {
24      // ถ้าจำนวนสระไม่เท่ากัน: เอาตัวที่มีสระมากกว่าขึ้นก่อน (B ลบ A)
25      return countB - countA;
26    } else {
27      // ถ้าสระเท่ากัน: เรียงตามตัวอักษร A-Z
28      return a.localeCompare(b);
29    }
30  });
31  // 4. นำ Array มาต่อกันเป็น String เดียว โดยคั่นด้วยช่องว่าง
32  let result = filteredStrings.join(" ");
33  return result;
34 }
35 // --- ตัวอย่างการใช้งาน ---
36 const strings = ["apple", "banana", "cherry", "date", "fig", "grape"];
37 console.log(processAndCombineStrings(strings, 5));
38 // Output: "BANANA APPLE GRAPE CHERRY"
39 console.log(processAndCombineStrings(strings, 6));
40 // Output: "BANANA CHERRY"
41 console.log(processAndCombineStrings(["hi", "hello", "world"], 2));
42 // Output: "HELLO WORLD HI"
```

test4.js

```
1 function analyzeStudentScores(students, passingScore) {
2   let passedStudents = [];
3   let totalScore = 0;
4
5   // 1. วงเล็บเพื่อคัดกรองคนสอบผ่านและรวมคะแนน
6   for (let student of students) {
7     if (student.score >= passingScore) {
8       passedStudents.push(student);
9       totalScore = totalScore + student.score; // รวมคะแนนเพื่อไปหาค่าเฉลี่ย
10    }
11  }
12
13  // 2. คำนวณค่าเฉลี่ย (ถ้าไม่มีคนสอบผ่านเลย ให้เฉลี่ยเป็น 0)
14  let average = 0;
15  if (passedStudents.length > 0) {
16    average = totalScore / passedStudents.length;
17  }
18
19  // 3. จัดเรียงจากคะแนนมากไปน้อย (Descending Order)
20  passedStudents.sort((a, b) => {
21    return b.score - a.score; // ถ้าตัวหลังลบตัวแรกได้ค่าบวก มันจะสลับเอาตัวมากไว้หน้า
22  });
23
24  // 4. ส่งผลลัพธ์กลับ
25  return {
26    passedList: passedStudents,
27    averageScore: average
28  };
29 }
30
31 // --- ตัวอย่างการใช้งาน ---
32 const studentData = [
33   { name: 'Alice', score: 45 },
34   { name: 'Bob', score: 80 },
35   { name: 'Charlie', score: 92 },
36   { name: 'David', score: 30 },
37   { name: 'Eve', score: 75 }
38 ];
```

```
43 /* Output ที่ได้:
44 {
45   passedList: [
46     { name: 'Charlie', score: 92 },
47     { name: 'Bob', score: 80 },
48     { name: 'Eve', score: 75 }
49   ],
50   averageScore: 82.33
51 }
52 */
```

mid\test6.js

```
1 function findLongestRepeatedSubstring(numbers) { // <--- รับค่าเดียวที่เป็น Array
2   let finalResult = [];
3   let groups = {}; // ใช้ Object เป็น "ถังเก็บของ" โดยมี Key คือ ผลรวมของหลักเลข
4
5   for (let num of numbers) {
6     // 1. ตรวจจับการหาผลรวมของหลัก (Digit Sum)
7     // เช่น 123 -> "123" -> ["1", "2", "3"]
8     let digits = num.toString().split('');
9     let sum = 0;
10
11     for (let d of digits) {
12       sum += parseInt(d); // บวกสะสม: 1 + 2 + 3 = 6
13     }
14     // 2. การจัดกลุ่มเข้า Object
15     // ถ้าในถังยังไม่มีช่องสำหรับผลรวมนี้ (เช่น เลข 6) ให้สร้าง Array วางรอไว้
16     if (!groups[sum]) {
17       groups[sum] = [];
18     }
19
20     // โยนตัวเลขต้นฉบับลงไปในช่องผลรวมที่คำนวณได้
21     groups[sum].push(num);
22   }
23   // 3. แปลง Object ให้เป็น Array ซ้อน Array (Nested Array)
24   for (let key in groups) {
25     finalResult.push(groups[key]);
26   }
27
28   return finalResult;
29 }
30 // --- วิธีเรียกใช้งานที่ถูกต้อง (ต้องใส่ [ ] ครบ) ---
31 const input = [123, 321, 213, 456, 654, 789];
32 console.log(findLongestRepeatedSubstring(input));
33 // Output: [ [123, 321, 213], [456, 654], [789] ]
```

METHOD อื่นๆ เพิ่มเติม

๓ การใช้งานแบบพื้นฐาน (แทนที่ค่าเดียว)

หากคุณส่งค่าเป็น "String" ปกติเข้าไป มันจะแทนที่เฉพาะ ตัวแรกที่เจอ เท่านั้นครับ

JavaScript



```
let text = "I love apple and apple.";
let result = text.replace("apple", "banana");

console.log(result);
// ผลลัพธ์: "I love banana and apple." (ตัวที่สองไม่โดนเปลี่ยน)
```

การใช้ Regular Expression (แทนที่ทั้งหมด)

ถ้าต้องการให้เปลี่ยน ทุกจุด ที่เจอในข้อความ ต้องใช้สิ่งที่เรียกว่า **Regex** พร้อมสัญลักษณ์ `/g` (Global) ครับ

JavaScript



```
let text = "I love apple and apple.";
let result = text.replace(/apple/g, "banana");

console.log(result);
// ผลลัพธ์: "I love banana and banana."
```

`.map()` : การแปลงร่างข้อมูล

ใช้เพื่อเปลี่ยนรูปแบบของสมาชิกแต่ละตัว (จำนวนสมาชิกจะเท่าเดิม แต่หน้าตาเปลี่ยนไป)

JavaScript



```
const filtered = ["elephant", "tiger"];

// เปลี่ยนทุกตัวเป็นตัวพิมพ์ใหญ่
const mapped = filtered.map(name => name.toUpperCase());

console.log(mapped);
// ผลลัพธ์: ["ELEPHANT", "TIGER"]
```

`.join()` : การรวมร่างเป็นข้อความ

ใช้เพื่อเปลี่ยนจาก Array (รายการ) ให้กลายเป็น String (ข้อความยาวๆ)

JavaScript



```
const sorted = ["ELEPHANT", "TIGER"];

// นำมารวมกันโดยคั่นด้วยช่องว่าง
const result = sorted.join(" ");

console.log(result);
// ผลลัพธ์: "ELEPHANT TIGER" (ไม่ใช่ Array แล้ว แต่เป็นข้อความเดียว)
```


METHOD อื่นๆ เพิ่มเติม

📌 ใช้กับข้อมูลประเภทอื่น (ต้องแปลงก่อน)

หากคุณมีข้อมูลประเภทอื่น แต่อยาก "เชื่อม" มันเข้าด้วยกัน คุณต้องเปลี่ยนให้มันเป็น Array ก่อนครับ:

- **String:** ถ้าอยากเอาคำมาแทรกระหว่างตัวอักษร ต้อง `split` ก่อน

JavaScript

```
const word = "HELLO";
const result = word.split('').join('-');
// ผลลัพธ์: "H-E-L-L-O"
```

- **Object:** ต้องดึงค่า (Values) หรือชื่อ (Keys) ออกมาเป็น Array ก่อน

JavaScript

```
const user = { name: "John", age: 25 };
const result = Object.values(user).join(' / ');
// ผลลัพธ์: "John / 25"
```

ตารางสรุปการใช้ `.join()`

ประเภทตัวแปร	ใช้ <code>.join()</code> ได้ไหม?	วิธีแก้ไข/วิธีใช้
Array	ได้ (ตรงรุ่น)	<code>arr.join('-')</code>
String	ไม่ได้	ต้องใช้ <code>.split('')</code> เพื่อเปลี่ยนเป็น Array ก่อน
Object	ไม่ได้	ต้องใช้ <code>Object.values()</code> หรือ <code>Object.keys()</code> ก่อน
Number	ไม่ได้	ต้องเปลี่ยนเป็น String หรือใส่ใน Array ก่อน

`.sort()` : การจัดเรียงลำดับ

ใช้เพื่อเปลี่ยนตำแหน่งของสมาชิกใน Array ตามเงื่อนไขที่เรากำหนด

JavaScript

```
const mapped = ["ELEPHANT", "TIGER"];

// เรียงลำดับตามตัวอักษร (A-Z)
const sorted = mapped.sort();

console.log(sorted);
// ผลลัพธ์: ["ELEPHANT", "TIGER"]
// (ถ้าเป็น ["TIGER", "ELEPHANT"] มาก่อน มันจะสลับที่กันใหม่)
```

`.some()` และ `.every()` : การ "ตรวจคัดกรอง"

เหมือนการเช็คเงื่อนไขก่อนส่งงาน หรือเช็คความถูกต้องของข้อมูลในแอปครับ

JavaScript

```
const scores = [85, 42, 90, 70];

// .some(): มีใคร "อย่างน้อยหนึ่งคน" สอบตก (คะแนนน้อยกว่า 50) ไหม?
const hasFailure = scores.some(s => s < 50);
console.log(hasFailure); // true (เพราะมี 42)

// .every(): "ทุกคน" สอบผ่าน (คะแนน 50 ขึ้นไป) ไหม?
const allPassed = scores.every(s => s >= 50);
console.log(allPassed); // false (เพราะมีคนได้ 42)
```

METHOD อื่นๆ เพิ่มเติม

`.slice()` : การ "ตัดแบ่ง"

เหมือนการตัดคำจากประโยคยาวๆ หรือการหยิบข้อมูลบางส่วนมาโชว์ที่หน้าจอแอป Flutter ครับ

JavaScript

```
let message = "Welcome to Lab";

// .slice(เริ่ม, จบแต่ไม่รวม): ตัดเอาแค่คำว่า "Welcome"
let word = message.slice(0, 7);
console.log(word); // "Welcome"

// .slice(เลขติดลบ): ตัดเอาจากข้างหลัง (เอา 3 ตัวสุดท้าย)
let lastPart = message.slice(-3);
console.log(lastPart); // "Lab"
```

การใช้ `.filter()` ร่วมกับ `.includes()`

เทคนิคนี้มักใช้ในการ **หาความซ้ำซ้อน** หรือ **การลบข้อมูล** ออกจาก Array ครับ

ตัวอย่าง: คุณมีรายการขยะทั้งหมด (All Items) และต้องการกรองเอาเฉพาะขยะที่เป็น "ขวด" (Bottles) ตามรายการที่คุณระบุไว้

JavaScript

```
const allItems = ["Apple", "Bottle", "Can", "Bottle", "Plastic"];
const targetItems = ["Bottle"];

// เลือกเฉพาะตัวที่มีอยู่ใน targetItems
const bottlesOnly = allItems.filter(item => targetItems.includes(item));

console.log(bottlesOnly); // ["Bottle", "Bottle"]
```

การใช้เพื่อ "ลบ" ข้อมูลที่ไม่ต้องการ

เพียงแค่เติมเครื่องหมาย **!** (Not) ไว้ข้างหน้า `.includes()` จะเป็นการบอกว่า "เอาทุกตัว ยกเว้น ตัวที่มีอยู่ในรายการนี้"

ตัวอย่าง: ลบตัวเลขที่ซ้ำกันออก (ประยุกต์ใช้กับโจทย์ข้อ 5)

JavaScript

```
const numbers = [1, 2, 3, 4, 5];
const duplicates = [2, 4];

// กรองเอาเฉพาะตัวที่ "ไม่อยู่ใน" duplicates
const uniqueNumbers = numbers.filter(num => !duplicates.includes(num));

console.log(uniqueNumbers); // [1, 3, 5]
```